Linux Driver I

開發學生: 溫方志、陳鎮國

中正大學 資訊工程學系 高效能計算實驗室

Linux Driver

- Linux Driver 簡介
- Linux Driver 種類
- 實驗目的
- Driver 範例
- 實驗過程

Linux Driver簡介（1）

- 介於應用程式與硬體裝置的軟體層，是這兩層溝通的橋梁
- 隱藏硬體device如何工作
- 提供一組 function 來代表硬體的運作

Linux Driver簡介（2）

- Driver提供一組標準的介面存取硬體，如此使用者的程式便不需要直接與硬體溝通。
- 當更換了不同的硬體，也只需載入不同的driver，不用重寫上層的應用程式。
- Driver為kernel space和user space間的interface。

Linux Driver種類

- Linux driver主要有三個種類，分別為字元、區塊、和網路這三個種類
- 其它還有USB、PCI等特殊的Driver
- 模組的分類並非硬性，也可以寫一個大模組能驅動以上三類裝置

實驗目的

- 寫一個 Driver 可以讓 kernel 掛載和卸載
- 了解 Driver 的基本開發過程和運作

簡單Driver範例（1）

- 我們首先向系統註冊一個driver，再向系統註冊我們所提供的open、close、read、和write的服務。
- **Initial module**

  當driver被載入之後第一個被呼叫的函式，類似一般C語言中的main function，在此function中向系統註冊為字元device和所提供的服務
- **Open device**

  當我們的device被fopen之類的函式開啟時，所執行的對應處理函式
- **Close device**

  使用者程式關閉我們的device時，執行的對應處理函式

簡單Driver範例（2）

- **I/O control**

  使用者可透過ioctl命令設定device的一些參數

- **Read device**

  當程式從我們的device讀取資料時，對應的處理函式

- **Write device**

  當程式對我們的device寫入資料時，對應的處理函式

- **Remove module**

  當driver被移除時所執行的處理函式，必須對系統取消註冊device

簡單Driver範例（3）

(hello.c)

```
#include <linux/init.h>
#include <linux/module.h>
#include <linux/kernel.h>
MODULE_LICENSE("Dual BSD/GPL");

static int demo_init(void) {
    printk("<1>I am the initial function!\n");
    return 0;
}

static void demo_exit(void) {
    printk("<1>I am the exit function!\n");
}
module_init(demo_init);
module_exit(demo_exit);
```

Makefile

```
obj-m := hello.o
all:
    make  -C  /[PATH]  M=$(PWD)  modules
clean:
    make  -C  /[PATH]  M=$(PWD)  clean
```

編譯模組

- 使用makefile編譯模組

  $ make ARCH=arm64 \
CROSS_COMPILE=/home/pschen/WORK/embedded_exp/buildroot/output/host/bin/aarch64-linux-

  這裡請適當修改成buildroot的路徑

  - 與編譯kernel image 時用的 cross compiler 相同

- make完之後就可以得到**hello.ko**這個module檔

Module 指令

- insmod Module_name.ko

  這個指令可以把模組碼和資料載入核心

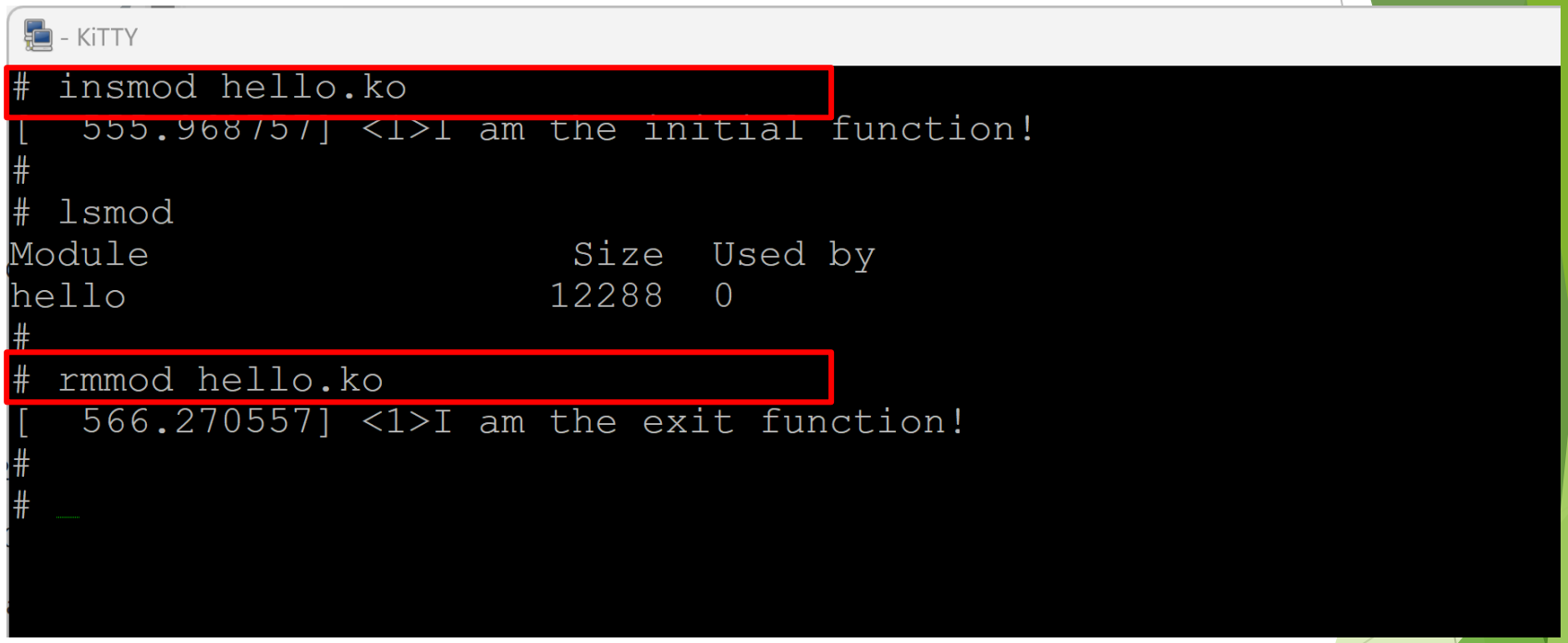
- rmmod Module_name.ko

  這個指令可以把模組碼和資料從核心卸載

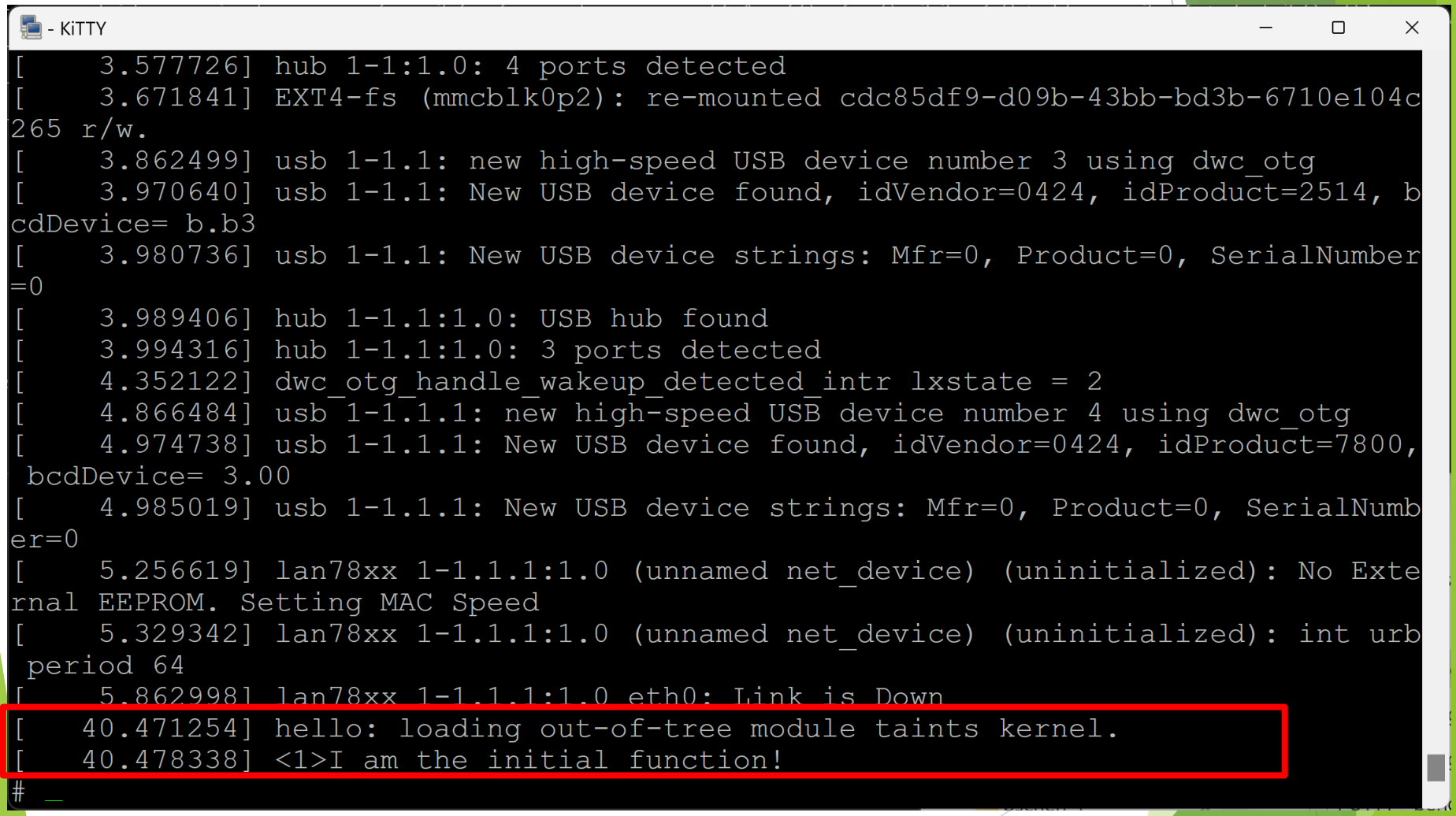
- lsmod，這個指令列出目前被使用的模組的狀態

測試實驗模組(1)

- 使用insmod和rmmod來掛載和卸載模組，使用lsmod查看模組



測試實驗模組(2)


- KiTTY
[3.577726] hub 1-1:1.0: 4 ports detected
[3.671841] EXT4-fs (mmcblk0p2): re-mounted cdc85df9-d09b-43bb-bd3b-6710e104c
265 r/w.
[3.862499] usb 1-1.1: new high-speed USB device number 3 using dwc_otg
[3.970640] usb 1-1.1: New USB device found, idVendor=0424, idProduct=2514, b
cdDevice= b.b3
[3.980736] usb 1-1.1: New USB device strings: Mfr=0, Product=0, SerialNumber
=0
[3.989406] hub 1-1.1:1.0: USB hub found
[3.994316] hub 1-1.1:1.0: 3 ports detected
[4.352122] dwc_otg_handle_wakeup_detected_intr lxstate = 2
[4.866484] usb 1-1.1.1: new high-speed USB device number 4 using dwc_otg
[4.974738] usb 1-1.1.1: New USB device found, idVendor=0424, idProduct=7800,
bcdDevice= 3.00
[4.985019] usb 1-1.1.1: New USB device strings: Mfr=0, Product=0, SerialNumb
er=0
[5.256619] lan78xx 1-1.1.1:1.0 (unnamed net_device) (uninitialized): No Exte
rnal EEPROM. Setting MAC Speed
[5.329342] lan78xx 1-1.1.1:1.0 (unnamed net_device) (uninitialized): int urb
period 64
[5.862998] lan78xx 1-1.1.1:1.0 eth0: Link is Down
[40.471254] hello: loading out-of-tree module taints kernel.
[40.478338] <1>I am the initial function!
_

完整的Driver

- 我們以最簡單的字元裝置為範例，將前面的hello world模組加入基本的open、close、I/O control、read和 write，就是一個簡單有完整功能的driver。

- 在 driver 的基本架構中，首先向系統註冊一個 driver，再向系統註冊我們所提供的 open、close、read 和 write的服務即可。

Driver範例（Part 1）

```
#include <linux/init.h>
#include <linux/kernel.h>
#include <linux/module.h>
#include <linux/fs.h>

MODULE_LICENSE("GPL");

static ssize_t drv_read(struct file *filp, char *buf, size_t count, loff_t *ppos)
{
    printk("device read\n");
    return count;
}

static ssize_t drv_write(struct file *filp, const char *buf, size_t count, loff_t *ppos)
{
    printk("device write\n");
    return count;
}

static int drv_open(struct inode *inode, struct file *filp)
{
    printk("device open\n");
    return 0;
}
```

Driver範例（Part 2）

```
static long drv_ioctl(struct file *filp, unsigned int cmd, unsigned long arg)
//2.6.36 version modify
{
    printk("device ioctl\n");
    return 0;
}

static int drv_release(struct inode *inode, struct file *filp)
{
    printk("device close\n");
    return 0;
}

struct file_operations drv_fops =
{
    .read=drv_read,
    .write=drv_write,
    .unlocked_ioctl=drv_ioctl,
    .open=drv_open,
    .release=drv_release,
}
```

Driver範例（Part 3）

```
#define MAJOR_NUM 60
#define MODULE_NAME "DEMO"

static int demo_init(void) {
    if (register_chrdev(MAJOR_NUM, "demo", &drv_fops) < 0) {
         printk("<1>%s: can't get major %d\n", MODULE_NAME,    MAJOR_NUM);
        return (-EBUSY);
    }

    printk("<1>%s: started\n", MODULE_NAME);
    return 0;
}
 static void demo_exit(void) {
    unregister_chrdev(MAJOR_NUM, "demo");
    printk("<1>%s: removed\n", MODULE_NAME);
}

module_init(demo_init);
module_exit(demo_exit);
```

使用 Makefile 編譯

使用Part1的makefile編譯Driver即可

```
obj-m := hello.o
all:
    make -C /[PATH] M=$(PWD) modules
clean:
    make -C /[PATH] M=$(PWD) clean
```

字元裝置

- 字元裝置可以讓應用程式像操作檔案一樣來存取裝置，每次傳輸一個byte，通常這類裝置至少需要實作open、close、read、write四個系統呼叫。

- 相對而言區塊裝置 (block device)，是指可以儲存資料的裝置，例如：硬碟、光碟機等等，每次傳輸整個區塊的資料，區塊的大小通常是512 bytes、或更大的2的次方大小。

建立裝置節點

- mknod /dev/demo c 60 0

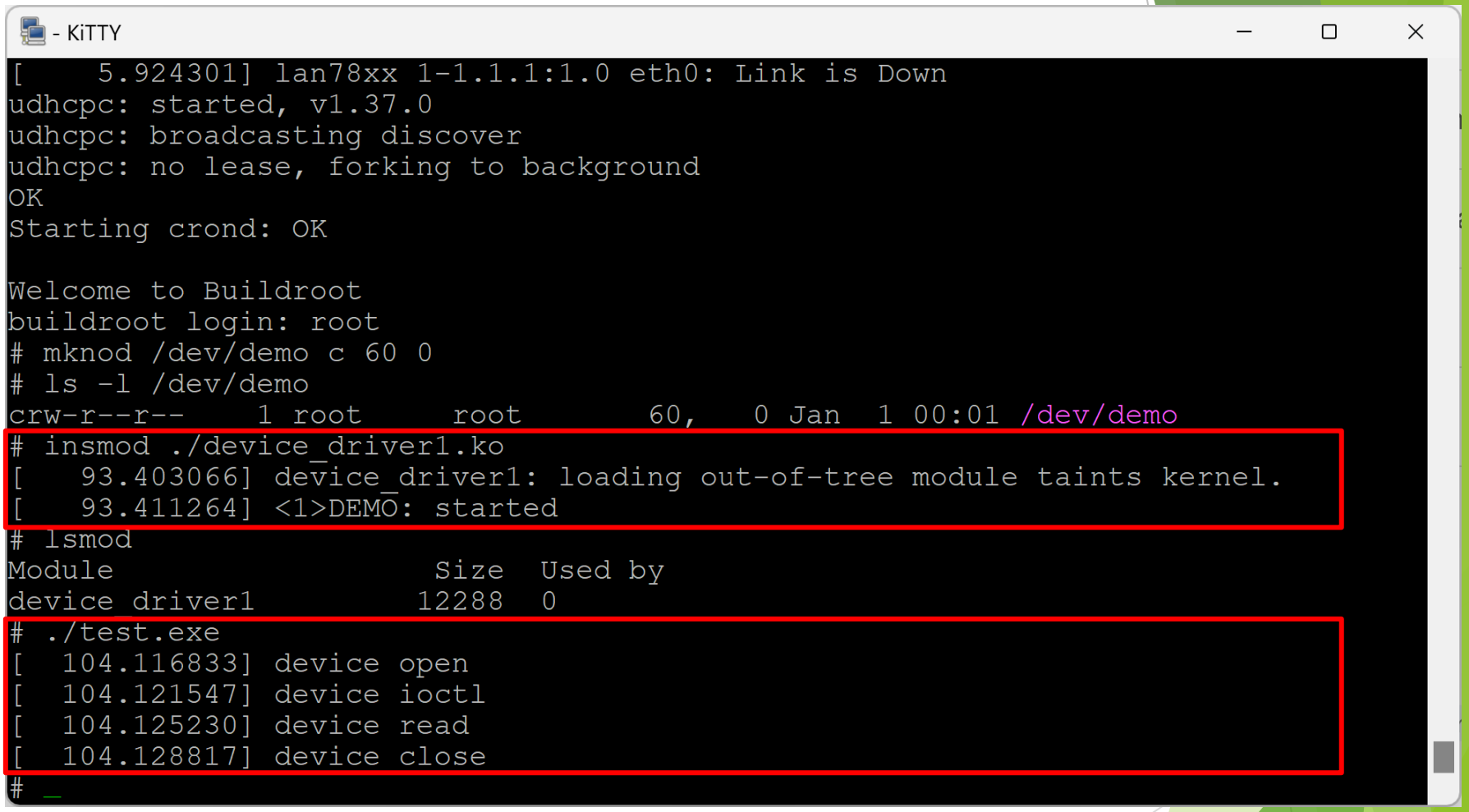
  其中/dev/demo 是裝置名稱，c 代表字元裝置，60 代表主要版本，0 代表次要版本

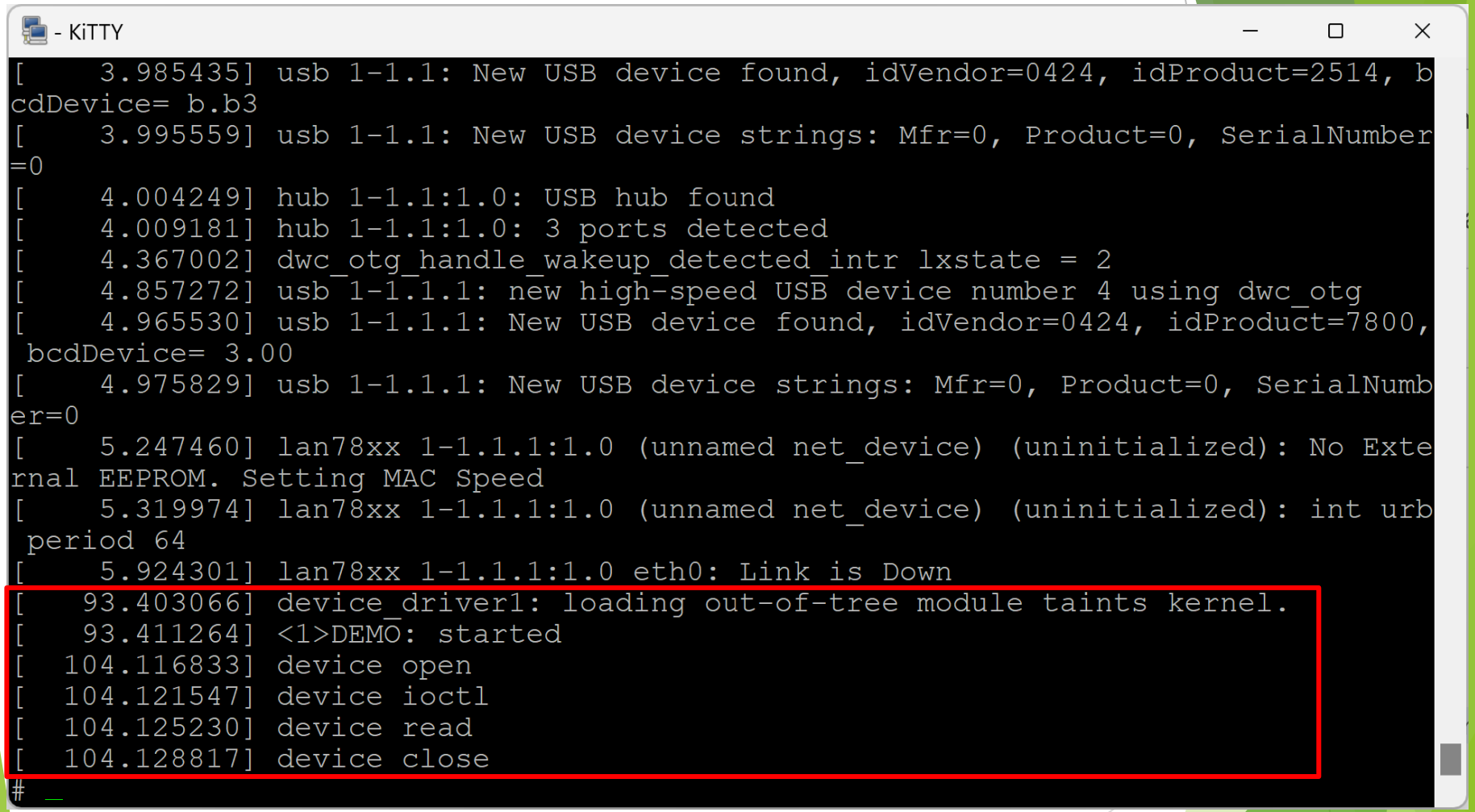
- 當 driver 在向系統註冊的時候也必須用同樣的型態、名稱和主要版本註冊，以免失敗。

Demo實驗步驟

- insmod把driver掛載上去
- 寫一個簡單的測試程式(test.c)來檢驗driver是否正常運作（請參閱詳細實驗手冊）
- 編譯此測試程式

  $ aarch64-linux-gcc -static -g test.c –o test.exe
- 檢視 driver 的訊息輸出結果


- KiTTY
[5.924301] lan78xx 1-1.1.1:1.0 eth0: Link is Down
udhcpc: started, v1.37.0
udhcpc: broadcasting discover
udhcpc: no lease, forking to background
OK
Starting crond: OK
Welcome to Buildroot
buildroot login: root
mknod /dev/demo c 60 0
ls -l /dev/demo
crw-r--r-- 1 root root 60, 0 Jan 1 00:01 /dev/demo
insmod ./device_driver1.ko
[93.403066] device_driver1: loading out-of-tree module taints kernel.
[93.411264] <1>DEMO: started
lsmod
Module Size Used by
device_driver1 12288 0
./test.exe
[104.116833] device open
[104.121547] device ioctl
[104.125230] device read
[104.128817] device close
#


- KiTTY
[3.985435] usb 1-1.1: New USB device found, idVendor=0424, idProduct=2514, bcdDevice= b.b3
[3.995559] usb 1-1.1: New USB device strings: Mfr=0, Product=0, SerialNumber=0
[4.004249] hub 1-1.1:1.0: USB hub found
[4.009181] hub 1-1.1:1.0: 3 ports detected
[4.367002] dwc_otg_handle_wakeup_detected_intr lxstate = 2
[4.857272] usb 1-1.1.1: new high-speed USB device number 4 using dwc_otg
[4.965530] usb 1-1.1.1: New USB device found, idVendor=0424, idProduct=7800, bcdDevice= 3.00
[4.975829] usb 1-1.1.1: New USB device strings: Mfr=0, Product=0, SerialNumber=0
[5.247460] lan78xx 1-1.1.1:1.0 (unnamed net_device) (uninitialized): No External EEPROM. Setting MAC Speed
[5.319974] lan78xx 1-1.1.1:1.0 (unnamed net_device) (uninitialized): int urb period 64
[5.924301] lan78xx 1-1.1.1:1.0 eth0: Link is Down
[93.403066] device_driver1: loading out-of-tree module taints kernel.
[93.411264] <1>DEMO: started
[104.116833] device open
[104.121547] device ioctl
[104.125230] device read
[104.128817] device close
_

Q & A

Q: 請問在撰寫Linux driver時，以下的巨集是做什麼用的?

- **MODULE_LICENSE()**
- **MODULE_DESCRIPTION()**
- **MODULE_AUTHOR()**

請於實驗報告裡回覆問題。